



Layout-Agnostic MPI Abstraction for Distributed Computing in Modern C++

Jiří Klepl^(✉) , Martin Kruliš , and Matyáš Brabec

Charles University, Malostranské náměstí 25, 118 00 Praha 1, Czech Republic
{klepl,krulis,brabec}@d3s.mff.cuni.cz

Abstract. Message Passing Interface (MPI) has been a well-established technology in the domain of distributed high-performance computing for several decades. However, one of its greatest drawbacks is a rather ancient pure-C interface. It lacks many useful features of modern languages (namely C++), like basic type-checking or support for generic code design. In this paper, we propose a novel abstraction for MPI, which we implemented as an extension of the C++ Noarr library. It follows Noarr paradigms (first-class layout and traversal abstraction) and offers layout-agnostic design of MPI applications. We also implemented a layout-agnostic distributed GEMM kernel as a case study to demonstrate the usability and syntax of the proposed abstraction. We show that the abstraction achieves performance comparable to the state-of-the-art MPI C++ bindings while allowing for a more flexible design of distributed applications.

Keywords: Distributed computing · Memory layout · Layout agnosticism · Noarr · C++ · Abstraction

1 Introduction

Distributed computing is one of the essential ingredients of high-performance parallel computing. MPI (Message Passing Interface) [8] represents one profound standard for this task, and it has many implementations, such as Open MPI [17] or MPICH [9]. Unfortunately, the MPI interface is rather old (considering the language development, especially in the past 25 years) and was designed on principles that are considered obsolete in modern C++, such as using `void*` pointers for generically typed buffers. Furthermore, optimizing HPC algorithms often requires the utilization of multiple fine-tuned memory layouts for the data structures, which is somewhat clumsy in MPI since it requires tedious manual construction of MPI datatypes for individual layouts. This makes any support for layout-agnostic data structures or traversal-agnostic index spaces challenging.

Layout agnosticism [16] separates the logical index space of a data structure (like an abstraction or an interface) from its physical memory organization

(implementation). A layout-agnostic data structure (e.g., a matrix) defines only its logical index space (such as the indices for the column i and the row j) and provides an external mechanism (a layout definition) for mapping the logical indices to physical memory locations (offsets in a linear buffer). The programmer may choose different layout definitions to optimize the performance of the data structure for a given problem or specific hardware (memory) architecture while the algorithm that uses the data structure remains unchanged. A few examples of matrix memory layouts are depicted in Fig. 1.

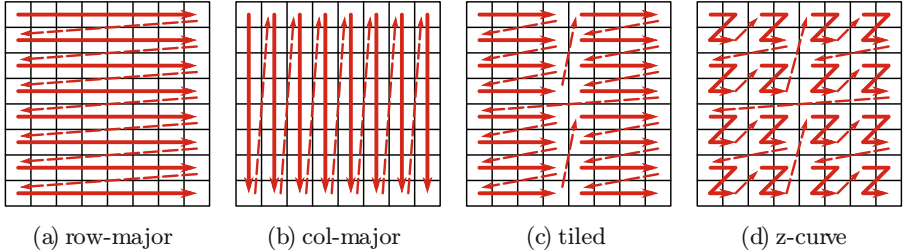


Fig. 1. Examples of common matrix layouts

The selected physical layout may affect the performance significantly. For instance, when multiplying two matrices $A \times B$ using a naïve $\mathcal{O}(N^3)$ algorithm, the first matrix A is accessed in a row-wise pattern while the second B is read column by column. Therefore, matrix A could benefit from the row-major layout, while the column-major format would be better for B .

Unfortunately, selecting the best layout for a given problem/architecture is not always possible since a suboptimal data layout may be enforced by the API, and internal transformation of the data structure may be computationally demanding. However, a similar approach can be used to design traversal-agnostic algorithms [7] where iteration over an index space is abstracted, and the loops may be reordered, split into blocks, or even parallelized without changing the algorithm itself or the involved structures. Continuing with our matrix multiplication example, the three loops iterating over the i, j, k dimensions (where i, j represent an element in the output matrix and k iterates over its dot product elements) can be split into blocks (in six nested loops) so the matrices are processed in a tiled manner, which improves data locality for most matrix layouts.

In recent years, there has been a growing interest in zero-cost overhead layout-agnostic data structures such as `std::mdspan` [18] included in the C++23 standard [6] (designed by Kokkos), Kokkos views [19], CuTe layouts [11] by NVIDIA, or Noarr structures [7]. The traversal agnosticism has also been tackled from various standpoints. At the lowest level, the C++ compiler attempts to provide automated loop optimizations based on the polyhedral model [5, 13]. However, the compiler sometimes has a limited perspective of the semantics of the algorithm, so various extensions (like Loopy [10]) were created that allowed the programmer

to guide the compiler’s loop transformations. An alternative approach is to use an abstraction layer for the data structure traversal. This may be achieved in a domain-specific language (DSL) such as Halide [15] or natively in C++ using meta-template programming (e.g., with Noarr traversers [7]).

When designing a fine-tuned HPC solution, a programmer must pay attention to all aspects affecting efficiency, which involves combining low-level code optimizations, memory layout optimizations, parallelization, and distributed computing. Particularly in the context of MPI applications, the data distribution handled by MPI communication primitives needs to work in synergy with layout-agnostic and traversal-agnostic program design and with (potential) layout transformations to ensure optimal efficiency. Finally, the layout definitions should be propagated in the datatypes to ensure basic type safety, and these types need to be translated into MPI datatypes to ensure correct data transfers.

There are several libraries that provide MPI abstractions that may be considered for the layout-agnostic and traversal-agnostic approach. For example, Boost.MPI can handle direct transfers of STL vectors and serialization of many other data structures. Some works, such as MPP or KaMPIng [12, 20], achieve similar goals with even better performance. Although these libraries offer some level of type safety and convenience when transferring simple containers, they are not designed for uses where the data needs to be also transformed to a different layout. KokkosComm [1] allows the user to send or receive an arbitrary Kokkos view that represents an mdspan. However, its public interface assumes that the sender and receiver layouts are the same and unpredictably reorders the data for seemingly compatible Kokkos views, even with the same value type, rank, and dimension extents. This makes the library incompatible with the layout-agnostic approach without modifications to its interface.

We propose the Noarr-MPI library, a novel MPI abstraction that adopts principles and technologies from Noarr [7], a C++ library for managing data structure memory organization and traversal. We have implemented a prototype that handles basic wrapping and necessary interactions between Noarr and MPI libraries, automatic data transformations, initialization of MPI structures, and type checking. It also provides an MPI abstraction compatible with Noarr traversers. The main contributions of the paper are:

- We propose a novel layout-agnostic, type-checked abstraction for MPI, designed for modern C++ and compatible with paradigms presented in the Noarr library.
- We provide an open-source prototype as an extension of the Noarr library¹.
- We compare the features and performance of the Noarr-MPI library with state-of-the-art MPI abstractions, demonstrating the advantages of our approach.
- We demonstrate the proposed abstraction on a well-known matrix multiplication (GEMM) kernel, which serves as a practical example and provides a basis for comparison with other abstractions.

¹ The implementation is available at <https://github.com/jiriklepl/noarr-mpi>.

The paper is organized as follows. Section 2 introduces the Noarr library and related work. Section 3 discusses mapping Noarr structures to MPI datatypes and the type transformation process. The proposed implementation is detailed in Sect. 4. Section 5 evaluates the applicability of the proposed approach on a real-world example and presents the comparison with other abstractions. Section 6 concludes the paper.

2 Noarr Background

Our work is built on top of the Noarr library [7], so we start with a brief description of its main principles essential for our MPI abstraction.

Noarr structures are the core abstraction of the Noarr library. They represent a mapping from the index space with named dimensions to the linear memory space. Structures are assembled from *proto-structure* objects that represent specific transformations of the mapping. For example, `vector<'i'>(N)` introduces a dimension named `'i'` with size `N`, and `into_blocks<'i', 'b'>(Ns)` splits dimension `'i'` into two dimensions where `'b'` is an index of a contiguous block of `Ns` elements and `'i'` represents individual elements within a block.

Multiple proto-structure objects can be combined using the `^` operator—e.g., `scalar<int>() ^ vector<'i'>(N) ^ vector<'j'>(M)` defines a 2D structure with dimensions named `'i'` and `'j'` of sizes `N` and `M`, respectively, and a scalar type of `int`. This composition is hierarchical, meaning that the `scalar<int>()` is the base structure further transformed by `vector<'i'>(N)` and finally by `vector<'j'>(M)`.

If `'i'` represents the rows of a matrix and `'j'` represents the columns, the `scalar<int>() ^ vectors<'i', 'j'>(N, M)` (a shorthand for the above) structure represents the column-major layout, while for the row-major we simply swap the dimension names: `scalar<int>() ^ vectors<'j', 'i'>(M, N)`.

Each Noarr structure has a **signature** that specifies the order of the dimensions in the structure. For the `scalar<int>() ^ vectors<'i', 'j'>(N, M)` structure, the associated signature can be represented as $j \rightarrow i \rightarrow \text{Int}$. The signature becomes important when iterating over the structure, as it defines the default traversal order—in the example, dimension `'j'` would be associated with the outermost loop.

The signature can be reordered using the **hoist** proto-structure that shifts a dimension to the front (outermost) position. For example, `hoist<'i'>` applied to the above structure would change the signature (and the default traversal order) to $i \rightarrow j \rightarrow \text{Int}$. The proto-structures change the signatures according to simple rewrite rules. For example, `into_blocks<'i', 'b'>(Ns)` replaces the appearance of `i` in the signature with $b \rightarrow i$, resulting in the signature $j \rightarrow b \rightarrow i \rightarrow \text{Int}$ if applied to the original structure. Besides the traversal order, the signatures also ensure the type safety of the transformations.

Noarr bags are smart pointers pairing Noarr structures with data buffers. They simplify the use of structures in code where the layout-agnostic data structure is usually materialized in memory. A `bag[idx<'i', 'j'>(i, j)]` expres-

sion accesses the element with index (i, j) regardless of the actual memory layout. Bags are designed to support both owning and observing semantics.

Noarr traversers are objects representing a traversal order over an index space of a Noarr structure or multiple structures. When constructed, they follow a combination of the default traversal orders of the given structures (prioritizing from the left). Traversers can be transformed by applying *proto-structures* to change the intended traversal order (similar to the construction of Noarr structures). For example, **hoist** changes the iteration order of the dimensions, and **span** restricts the iteration space along a given dimension. To ensure semantic correctness, proto-structures that change the physical layout (such as **vector** that expands the memory layout) are not allowed for traversers; however, some of them have corresponding counterparts for traversers. For example, **vector** can be replaced with **bcast** that introduces a new dimension to the traversal (a new loop) without implying a change in the physical layout.

Given a set of Noarr bags and a Noarr traverser, the user can define a lambda function representing a computation and combine it with the traverser via the `|` operator, as demonstrated in Listing 1. The traverser applies the lambda function to each element of the (possibly transformed) iteration space represented by a **Noarr state** object containing the indices of a given point in the iteration space (such as `idx<'i', 'j'>(i, j)`) that can be used to access the corresponding elements of the Noarr bags.

```
// Allocate bags for the matrices with the given layouts:
auto C = bag(scalar<int>()) ~ vector<'i'>(M) ~ vector<'j'>(N));
auto A = bag(scalar<int>()) ~ vector<'i'>(M) ~ vector<'k'>(K));
auto B = bag(scalar<int>()) ~ vector<'k'>(K) ~ vector<'j'>(N));

traverser(C) | [&](auto state) { // C provides iterations space over (i,j)
    C[state] = 0;
    traverser(A, B) ~ fix(state) | [&](auto state) { // iterating over k (i,j were fix-ed)
        C[state] += A[state] * B[state]; // [] applies relevant index sub-set of state
    };
};
```

Listing 1. Naïve matrix multiplication kernel in Noarr

Noarr also provides constructs to parallelize the traversal of selected dimension(s) based on existing parallel libraries like OpenMP (CPU) or CUDA (GPU).

2.1 Related Work

Various libraries and tools have been developed to make the use of MPI more convenient and less error-prone in the context of C++ programming. We briefly discuss the most relevant libraries in the context of implementing layout-agnostic MPI abstractions. Most of these works have additional (orthogonal) goals, such as handling errors in MPI communication, or abstracting the non-blocking primitives to C++ promises and futures.

As the oldest, still relevant work, the Boost.MPI [4] library provides a higher-level interface for C++ STL containers and user-defined types in MPI communication by (de)serializing the data. It introduces a form of layout agnosticism

to MPI via a *skeleton* object, which describes the layout of a particular data structure. Although the skeleton approach is quite generic (covering even linked lists or trees), its creation still involves costly (de)serialization. Furthermore, after creating the skeleton, the user has to ensure that the data structure is unchanged and that it is not paired with an incompatible skeleton (reducing the type safety of the communication).

The MPP library [12] improves on Boost.MPI by avoiding (de)serialization. It uses `MPI_Type_create_struct` to define the layout of general data structures. It can also efficiently transform layouts of arbitrary C++ containers by analyzing their type traits (querying the starting memory address of the elements, their size, and the number of elements).

Another improvement over Boost.MPI was introduced by the MPL header-only library [2], which enables MPI communication of data structures by defining their *layout*, which can be built (semi)automatically for trivial C++ types and standard containers. The layouts map well to MPI datatypes; however, their construction for non-trivial layouts is cumbersome.

More recently, Demiralp et al. [3] presented a modern C++ library for MPI that uses compile-time reflection on C++ types to generate appropriate MPI datatypes automatically. It introduced useful features such as exception-based error handling; however, its container support is limited to array-like contiguous containers (e.g., `std::span` or `std::vector`).

The KaMPIng library [20] is the most recent major work that aims to introduce some layout agnosticism and type safety to MPI. It maps C++ types to MPI datatypes at compile time via type introspection using template meta-programming. It supports standard C++ containers and general trivially copyable types (which can be copied as contiguous sequences of bytes on homogeneous systems without needing to analyze their layout). However, it does not support non-contiguous containers, significantly limiting its usability in layout-agnostic applications.

The experimental KokkosComm library [1] is a recent addition to the Kokkos ecosystem that extends the Kokkos library [19] to support MPI communication. It is also the work that is closest to our Noarr-MPI abstraction in its general goal. Since Kokkos aims to maximize performance and portability, it offers some layout agnosticism via the `Kokkos::View` abstraction. It stands out as the only related library that supports sending and receiving non-contiguous multidimensional data automatically without requiring any data preprocessing (such as serialization or data packing). The authors show that using the automatically generated MPI Datatypes in communication performs on par with hand-written datatypes. However, the KokkosComm library is still in its early stages and does not sufficiently support the necessary primitives to implement a fully layout-agnostic MPI abstraction—most notably, it does not fully support scatter and gather operations.

None of the existing MPI-abstracting libraries provide a layout-agnostic abstraction that could automatically handle MPI communication involving multiple data layouts with different compatible physical layouts (e.g., row-major and

column-major matrices of the same size). This is a significant limitation for use cases that call for a different layout of the data structures on the sending and receiving sides, which may easily happen if the layouts on the two sides are tuned independently to achieve the best performance. Using the existing libraries, the user has to manually define fitting MPI datatypes, which is tedious and error-prone, or resort to data packing and unpacking, which introduces additional overhead and is not transparent to the MPI backend. The Noarr-MPI library aims to eliminate this limitation by providing a layout-agnostic abstraction for MPI communication that can automatically handle the definition of the MPI datatypes based on the Noarr structures and traversers.

3 Noarr and MPI Layout Mapping

For successful Noarr-MPI binding, we need to define a mapping from Noarr structures to MPI datatypes. This mapping is non-trivial due to philosophical differences between the two libraries. On the other hand, datatypes both in MPI and in Noarr are constructed hierarchically from simple base types, giving us a common ground for the mapping.

```
auto row = scalar<int>() ^ vector<'n'>(n);
auto matrix = row ^ vector<'m'>(m);

/* MPI: */ MPI_Type_contiguous(n, MPI_INT, &row);
           MPI_Type_contiguous(m, row, &matrix);
```

Listing 2. Constructing an integer matrix in MPI and Noarr

Listing 2 illustrates two semantically identical definitions of a matrix layout in both libraries, showcasing their superficial similarities. The derived datatypes are strictly made of copies or sequences (so `MPI_Type_contiguous` can be used). However, more complex data layouts (with tiling or slicing) will require more complex transformation.

```
auto matrix_tiled = matrix ^ into_blocks<'m', 'M'>(M) ^ into_blocks<'n', 'N'>(N);

/* MPI: */ MPI_Type_contiguous(N, MPI_INT, &tileRow);
           MPI_Type_contiguous(n / N, tileRow, &row);
           MPI_Type_contiguous(M, row, &slice);
           MPI_Type_contiguous(m / M, slice, &matrix);
```

Listing 3. Tiling the matrix structure

Listing 3 shows a tiled transformation of the `matrix` type from the previous example that constructs a layout with the same underlying physical data but logically tiled into blocks of size $M \times N$ where each block is indexed by `'M'` and `'N'` (in Noarr). The contiguous blocks are specified separately to match the logical structure of the data. If a different logical ordering of the elements is required, the user has to use functions such as `MPI_Type_create_hvector` and then carefully arrange the order of MPI calls and their arguments.

3.1 Type Transformation

Here, we present a general approach for transforming Noarr structures into MPI datatypes via the `mpi_transform` function. It is used internally by the Noarr-MPI bindings, but can also be used directly by the user to create an RAII datatype handle that automatically releases the MPI datatype when it goes out of scope.

```
auto matrix_tiled = matrix ^ into_blocks<'m', 'M'>(M) ^ into_blocks<'n', 'N'>(N);

auto datatype_handle = mpi_transform(matrix_tiled);
datatype_handle.commit();

MPI_Bcast(data, (MPI_Datatype)datatype_handle, root, comm);
```

The type transformation is defined recursively according to the dimension hierarchy encoded in the signature of the structure. For `matrix_tiled`, the signature is $M \rightarrow m \rightarrow N \rightarrow n \rightarrow \text{Int}$ (resulting from the signature $m \rightarrow n \rightarrow \text{Int}$ of `matrix`). Generally, a signature is a tree with the outermost dimension in the root and the resulting scalar types in leaves. For each dimension, the process first constructs the MPI types of the nested dimensions and then uses them to construct the MPI type of the current dimension.

The datatype construction at each given dimension is usually equivalent to a single MPI call. The resulting MPI datatype has to match the traversal over the Noarr structure along the given dimension; namely, each $(\text{type}_i, \text{displacement}_i)$ pair in the resulting MPI datatype has to match the types and relative offsets of all sub-items i . Based on the types and displacements, an appropriate MPI call is selected:

1. If the structure is stored contiguously and all types are the same (e.g., in the case of Noarr `vector`), the `MPI_Type_contiguous` can be used to construct the MPI datatype. This is the case for `matrix_tiled` in Listing 3.
2. If the structure is not stored contiguously but still follows constant strides, `MPI_Type_create_hvector` can be used.
3. If the offset of the first element is non-zero or the strides are not constant, we can use `MPI_Type_create_hindexed` to specify offsets for individual items.
4. If the structure is not uniform along the given dimension (i.e., $\text{type}_i \neq \text{type}_{i'}$ for some $i \neq i'$), the `MPI_Type_create_struct` can be used to create irregular MPI structures.

Noarr `get_length` and `get_size` functions are designed to determine the index ranges and the extents of the sub-structures. Additional trait-like functions, such as `is_uniform_along`, `stride_along`, and `lower_bound_along`, can be used to select the appropriate MPI call.

Figure 2 showcases the construction from Listing 2 disregarding the contiguity of the structure to illustrate the parametrization of the MPI calls. The block counts can be determined using `get_length`, and the stride parameters can be computed using `stride_along`.

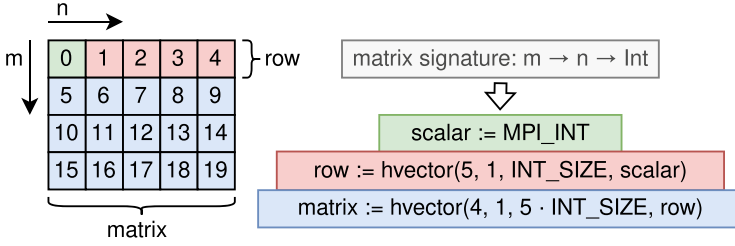


Fig. 2. Type transformation from Noarr structures to MPI datatypes

3.2 Layout-Agnostic Type Transformation

Our main objective is to enable the transfer of data structures with identical index spaces but different physical layouts. This will require data transformation during the transfer, which can be achieved seamlessly with the MPI communication primitives as long as the MPI datatypes are constructed properly.

To build on the matrix example, let us consider a broadcast operation where the source and destination structures are column-major and row-major matrices, respectively. Naïvely, we might construct a contiguous MPI datatype for each matrix, which would be efficient; however, it would also corrupt the data on transfer due to differing orderings of elements between the two structures, which is the case for frameworks like KokkosComm [1]. We need to ensure that the MPI datatypes are compatible and that they correctly represent the given data structures at the same time.

There are multiple ways to construct compatible MPI datatypes; perhaps the simplest one would be to reorder the signatures to a canonical form (e.g., lexicographically ordered dimensions). However, this can result in suboptimal transfers (the performance depends on how the layouts are described, not just on the layouts themselves), and it is impractical to affect efficiency by a mere choice of dimension names. To avoid this, a traverser can be provided to define the intended hierarchy of dimensions during the transformation via its signature. Since the traverser has the same index space as its Noarr structure parameters, we can construct it from the list of the structures involved in the communication to ensure correctness. Furthermore, the user may transform the traverser for the purpose of performance tuning (e.g., apply `noarr::hoist` to reorder the dimensions).

Constructing the MPI datatype for `matrix` according to the default traverser of the transposed matrix differs from the construction in Fig. 2 only in the dimension hierarchy (switching the order of `hvector` calls), which changes the order of the elements in the resulting MPI datatype. However, unlike in Listing 2, the `hvector` calls cannot be replaced with `contiguous`. The resulting MPI datatype is compatible with `MPI_Type_contiguous` over the memory space of the transposed matrix, enabling transfers between the two structures without data corruption.

4 MPI Interface Abstraction

Our MPI interface extends the Noarr traverser abstraction (for iterating over an index space) into an *MPI traverser* that combines a regular traverser with an MPI communicator, which enables a seamless extension of existing Noarr algorithms.

4.1 MPI Traverser

MPI traversers abstract iteration over an index space distributed over multiple ranks (MPI processes) in the same group (MPI communicator). They are inspired by Noarr CUDA traversers [7], which abstract work distribution over a CUDA thread grid (running on a GPU).

The construction of MPI traversers is shown in Listing 4. The constructor selects one dimension as the *ranking dimension* (`'r'`), and its index is bound to the MPI rank. The size of the ranking dimension must match the size of the communicator, and it is set automatically to this size if not specified explicitly. The resulting MPI traverser can be used as a regular Noarr traverser and in place of an MPI communicator in Noarr-MPI bindings (communication operations).

```
auto trav = traverser(matrix ^ into_blocks<'m', 'r', 's'>()); // naming the inner dim. 's'
auto MPITrav = mpi_traverser<'r'>(trav, MPI_COMM_WORLD);
auto slice = bag(scalar<float>() ^ vectors_like<'s', 'm'>(MPITrav)); // copy dims of MPITrav

if (mpi_get_comm_rank(MPITrav) == 0) {
    // Using `MPITrav` as a Noarr traverser (applying a lambda for execution)
    MPITrav | [&](auto state) { slice[state] = init(); };
}
// Using `MPITrav` as an MPI communicator for broadcast operation
broadcast(slice, MPITrav, 0);
```

Listing 4. Defining an MPI traverser over the dimension `'r'`

In Listing 4, the matrix sized $m \times n$ is sliced along the `'m'` dimension into r sub-matrices $s \times n$ where $s = m/r$ and r is determined automatically from the communicator size. The root node initializes the slice and then broadcasts it to all nodes in the communicator associated with the traverser.

4.2 Collective Communication

Simple type transformation from Noarr structures to MPI datatypes is sufficient for broadcasts and point-to-point communication that use the same logical layout for both the sending and the receiving processes. However, more complex operations like scatter and gather involve data structures with not only different physical layouts but also different logical layouts—the index space of one input structure being a subspace of the other input structure. The scatter and gather are representative collective operations for our transformations, and since they are symmetric, we discuss only the scatter operation in detail.

A naïve approach to scattering of general layouts would involve costly data (de)serialization; however, this can be avoided with the Noarr library and the

type transformations described in Sect. 3.2, making the operation layout-agnostic thanks to constructing the MPI datatypes from the Noarr structures according to the logical ordering of elements dictated by the same traverser.

After constructing the MPI datatypes for the two input structures, the scatter operation uses the ranking dimension of the MPI traverser to determine the correct displacements of the sub-structures and scatters the data accordingly. Since the ranking dimension is bound to the MPI rank, it maps to the tiles of the source structure.

```
auto matrix = bag(matrix_tiled, buffer);

// We need to fix one block dimension (M=4) and associate r with blocking dimensions MxN
auto trav = traverser(matrix) ^ set_length<'M'>(4) ^ merge_blocks<'M', 'N', 'r'>();
auto MPITrav = mpi_traverser<'r'>(trav, MPI_COMM_WORLD);
// Specify the layout of the tile (with a matching index space)
auto tile = bag(scalar<float>() ^ vectors_like<'m', 'n'>(MPITrav));

if (mpi_get_comm_rank(MPITrav) == 0) {
    // Traverse `matrix` using the dimension lengths from `MPITrav`
    traverser(matrix) ^ set_length(MPITrav) | [&](auto state) { matrix[state] = init(); };
}
scatter(matrix, tile, MPITrav, 0);
MPITrav | [&](auto state) { modify(tile[state]); }; // Process the tile on each worker
```

Listing 5. Scatter operation using an MPI traverser

Listing 5 uses the tiled matrix from Listing 3 and scatters it across all processes where the tiles are processed by the `modify` function. The key aspect here is that we wish to divide the matrix into tiles so that each process handles one tile. However, dimensions `'M'` and `'N'` are both open, so we need to fix one of them to a specific value (in this case, `'M'` is fixed to 4) so the remaining dimension can be set $N = r/M$ based on the number of processes (which is ensured by the `merge_blocks` association). The $M \times N$ grid of tiles is distributed so that MPI processes with ranks $0 \dots N - 1$ compute row of tiles with `'M'` index 0, ranks $N \dots 2N - 1$ compute row where `'M'` is 1, and so on. Currently, our implementation requires that M is selected as a divisor of r , and original matrix dimensions (m, n) must be divisible by M and N , respectively.

Perhaps the greatest benefit is that the layout of the tiles (`tile` structure) can be defined independently of the layout of the original matrix. The scatter operation performs the necessary transformation to convert the layouts, making the tile layouts and the computation performed on them independent of the original matrix layout. Thus, the layout-agnostic design can fully exploit the memory architecture and caches of the target node.

The proposed abstraction also ensures the type safety of the operation. The index space of the distributed structure has to be a subspace of the root structure index space, and the difference in the index spaces has to be covered by the dimension bound to the MPI communicator. This eliminates the possibility of a mismatch between the input structures and simplifies the API.

5 Evaluation

We evaluate the proposed Noarr-MPI library by comparing it with selected MPI abstractions in terms of the supported features, ease of use, and performance of the generated code. In the evaluation, we include the MPI libraries introduced in Sect. 2.1, which demonstrate at least some overlap with our main objectives (layout agnosticism and type safety): **MPI** (using standard MPI 4.0 interface) with C++ mdspans, **Boost.MPI** [4] with C++ mdspans, **MPP** [12], **MPL** [2], **KokkosComm** [1], and **KaMPIng** [20]. Our proposed solution is denoted **Noarr-MPI**.

For the approaches that support a sufficient level of layout agnosticism, we perform a case study of implementing a distributed general matrix multiplication (GEMM) kernel as defined in the Polybench/C benchmark suite [14], with each MPI process computing one sub-matrix of the output matrix by multiplying the corresponding sub-matrices of the input matrices. We discuss the necessary steps to implement the GEMM kernel in Sect. 5.2 and evaluate the performance of the implementations in Sect. 5.3.

5.1 Feature Comparison

The comparison of the individual libraries focuses mainly on the features related to the layout agnosticism.

Automatic transformations of layouts if the source and destination processes use a different layout datatype specification. This is a crucial feature for layout agnosticism as it allows for defining the layout of each data structure independently without specifying explicit data transformations.

Non-contiguous layouts—i.e., support for data structures that are not laid in one compact block of memory. This is especially essential for scatter and gather operations where many sub-structures will be non-contiguous.

Mdspan-like representation for the data structures involved in the communication. It allows the user to reason about layouts in terms of the logical order of the elements in the data structure while the library automatically handles the construction of the MPI datatypes.

Seamless libraries avoid unnecessary data packing or serialization when transferring the data structures. This is not a hard requisite, but it may improve efficiency.

Type safety of a library indicates that it should fail to compile the code if the source and destination data structures have incompatible index spaces.

Scatter/gather operations of multi-dimensional data structures are a crucial feature for layout agnosticism. Without this feature, the user has to specify a linearized view of the data, which intricately depends on the data layout and introduces unnecessary complexity and potential errors.

Table 1 summarizes the features of the evaluated libraries. The ★ mark for the MPI support of auto-transformations denotes that these transformations are technically possible, but they require significant additional effort. The datatypes must be constructed imperatively, not automatically inferred, like in Noarr-MPI.

Table 1. Comparison of the MPI abstractions

	Noarr-MPI	native MPI	Boost.MPI	MPP	MPL	Kokkos	KaMPIng
Auto-transforms	✓	✱	✗	✗	✗	✗	✗
Non-contiguous	✓	✓	✓	✓	✓	✓	✗
Mdspan-like	✓	✗	✗	✗	✗	✓	✗
Seamless	✓	✓	✗	✓	✓	✓	✓
Type-safety	✓	✗	✓	✓	✓	✓	✓
Scatter/gather	✓	✓	✗	✗	✓	✗	✗

5.2 Programming Effort Discussion

Since conducting a comprehensive user study is beyond the scope of this paper, we provide a discussion of the necessary programming effort to implement the GEMM kernel using the libraries that support a sufficient level of layout agnosticism as described in Sect. 5.1 (supporting more than half of the discussed features). Using text-based or syntax-based metrics for this comparison would be misleading as the libraries differ significantly in their approach, and any practical layout-agnostic use of the libraries would be accompanied by support abstractions that would significantly reduce the amount of necessary boilerplate code. Instead, we focus on the necessary steps and required information that the user has to provide to implement the GEMM kernel using the evaluated libraries, disregarding the specific syntax or verbosity that could be hidden behind a shorthand abstraction.

An ideal tunable and generalizable implementation of the kernel would require only the *definition of the data layout* of each matrix and its privatized tile on a given worker process, the *scattering strategy*, the implementation of *the computation*, and some specification of *data dependencies* (in all the evaluated libraries, this is done via explicit calls representing MPI communication). For each of these general steps, we discuss the specific sub-steps that are needed to implement the GEMM kernel using the evaluated libraries.

Data Layout Definition. When using the Boost. MPI library or the standard MPI interface, we have used the `mdspan` abstraction designed by the Kokkos group and adopted in the C++23 standard [18] to define the data layout of each matrix and its privatized tiles. Using `mdspans` is sufficiently expressive and convenient for layout agnosticism. They are compatible with the libraries and do not introduce any unnecessary overhead. For the KokkosComm library, we use Kokkos views, which are very similar to the `mdspan` abstraction and the Kokkos library uses them natively.

Both of these approaches require the user to define the index ranges of each sub-matrix tile separately (and match them correctly), which creates an unnecessary source of potential errors. Noarr structures in the Noarr-MPI approach allow us to specify the index ranges collectively for all tiles. Additionally, the

library can automatically deduce the ranges of the tiles (e.g., based on the number of MPI processes).

Scattering Strategy. Each library has its own way of defining how to scatter and gather the data. The Noarr-MPI library infers the correct MPI datatypes from the input and output data structures (Sect. 3.2). The inferred datatypes can be further tuned (for optimizing transformations) by reordering the logical order (hierarchy) of the dimensions of the given traverser (Sect. 4.1). Noarr-MPI also enables the user to easily specify the mapping of the tiles to the worker threads.

In the approach that uses just the standard MPI interface, we build the MPI datatypes by iteratively using the `MPI_Type_create_hvector` function with the extent and the stride of each dimension of the given `mdspan` from the rightmost to the leftmost (we have experimentally verified that this is the most efficient order for this use case; the supporting experiments follow the methodology described in Sect. 5.3 and are included in the replication package). This generalizes well to an arbitrary number of dimensions; however, this approach may be suboptimal for other use cases.

The KokkosComm library internally uses the same approach; however, its API does not support scattering and gathering. They offer non-blocking receive and send operations that can be used to emulate the scatter and gather operations with very little overhead but at much higher programming effort. Since the library does not consider layout agnosticism, its receive and send interface is unusable for our purposes (it works inconsistently for contiguous and non-contiguous data layouts). However, the library does publicly expose the necessary building blocks for layout-agnostic send and receive operations.

The Boost.MPI library does not support the scatter and gather operations natively for this use case, so we have opted to try two different approaches: the first one is to define wrappers that specify how the items of a `mdspan` should be serialized and deserialized via a simple iteration in the logical order of the elements and scatter this serialized data. The other approach is to use input and output archives in non-blocking send and receive operations; however, the second approach performed consistently worse, so it is not included in the evaluation. Using skeletons in the Boost.MPI is too cumbersome for this use case and is not included in the evaluation.

Computation Kernel Implementation. The computation kernel is implemented similarly across all libraries and is rather straightforward (almost verbatim copy of the GEMM loop nest from the PolyBench/C benchmark).

The only outlier is the Noarr-MPI library, which uses Noarr traversers. As analyzed in our previous work [7], the traverser abstraction significantly reduces the amount of explicit indexation, which can prevent potential errors and makes the code more flexible for possible tuning. In this aspect, the Noarr library is similar to `std::ranges` in C++20, which also allows for a more declarative design of algorithms. Even with a slight syntactical difference between the Noarr

traversers and an explicit loop nest, all libraries follow the same conceptual pattern of the computation kernel and specifying it via a loop nest can be directly 1-to-1 mapped to Noarr traversers.

5.3 Performance Evaluation

In the performance evaluation, we measure only the code that would be executed repeatedly in a real-world application (i.e., the `scatter`, `compute`, and `gather` operations). We do not include the initialization of the MPI environment, allocation of the buffers, data validation, or cleanup operations. To ensure a fair comparison, all implementations allocate the data the same way, and all libraries use views with non-owning semantics and without any additional data movements except those necessary for scattering and gathering the data. Most importantly, all implementations perform the exactly same computation in the same order and on the same data layouts. We have also ensured that all implementations follow the same data distribution strategy (i.e., send/receive the same data on each given process). The details and all measured data can be found in our replication package².

The evaluation was performed on a Slurm-managed cluster with 8 nodes interconnected via InfiniBand FDR (56 Gb/s), each with two sockets equipped with Intel Xeon Gold 6130 processors, each comprising 2×16 hyper-threads. All GEMM implementations were compiled with GCC 14.2 and Open MPI 5.0.6 using the `Release` configuration as defined by CMake 3.27. The evaluation used the various datasets from the Polybench/C benchmark, which were modified to be divisible by the number of MPI processes; however, we visualize only the two opposite ends of the spectrum, `MINI` and `EXTRALARGE` (all dataset results are available in the replication package), which show interesting performance trends. For the `MINI` dataset, all dimensions are 64, and for the `EXTRALARGE` dataset, the dimensions are $n_i = 2048$, $n_j = 2560$, $n_k = 1408$. The performance is measured in seconds by a wall clock, and each visualized result is the average and standard deviation of 100 runs of the distributed GEMM kernel.

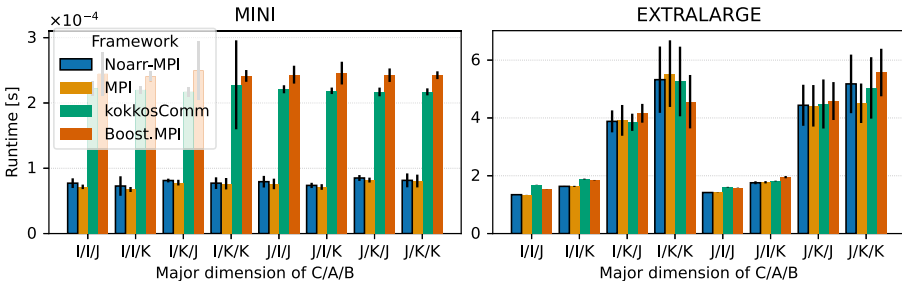


Fig. 3. Performance evaluation of the evaluated libraries (columns show mean runtime in seconds, error bars show standard deviation)

² The replication package is available at <https://github.com/jiriklepl/noarr-mpi>.

Figure 3 shows the results of the performance evaluation. The individual configurations (labeled on the x-axis) represent the major dimensions of the privatized tiles used in GEMM for the **C**, **A**, and **B** matrices written as $C/A/B$ (using dimension names from Listing 1). For example, the **I/I/J** configuration denotes that the **C** and **A** matrix tiles are laid out in a row-major order and the **B** matrix tiles in a column-major order (**C** tiles are $I \times J$ matrices, **A** tiles are $I \times K$, and **B** tiles are $K \times J$). All implementations passed the validation checks.

The measurements performed on the **MINI** dataset show that the **Noarr-MPI** library performs on par with the standard MPI interface for C++, while the other approaches are significantly slower. The overheads on this dataset are dominated by the non-computational parts of the code (i.e., communication latency, datatype construction, and data movements during scattering). The **SMALL** and **MEDIUM** datasets show a similar trend as the **MINI** dataset, but the differences between the libraries are less pronounced.

In the case of the **EXTRALARGE** dataset, computational time significantly dominates the communication overhead. The less efficient configurations show a significant variation in performance due to inefficient memory accesses. For most configurations, the **Noarr-MPI** library performs on par or slightly better than the other libraries. The only significant outliers that show a performance drop when using the **Noarr-MPI** library are the **I/K/K** and **J/K/K** configurations, where the serialization strategy of the **Boost.MPI** implementation outperforms the other libraries, which defer the layout transformation to the MPI backend. However, for all four most efficient configurations, the **Noarr-MPI** library performs the best. The **LARGE** dataset shows a similar trend, but **Noarr-MPI** (and other libraries) do not experience the performance drop on the four less efficient configurations.

The comparison shows that the proposed **Noarr-MPI** library can achieve performance comparable to the state-of-the-art MPI C++ bindings while allowing for a more flexible design of distributed applications and thus improving the performance without expending the programming effort necessary to move the data correctly. Except for a few outliers, its performance is equivalent to hand-written MPI code using the C++ `mdspan` abstraction for computation.

6 Conclusion

Noarr-MPI is a novel C++ abstraction for MPI that introduces a layout-agnostic design to distributed computing. It builds on top of the **Noarr** library and closely follows its paradigms. We have used the abstraction to implement an MPI version of the GEMM kernel to demonstrate its applicability and performance compared to the state-of-the-art MPI C++ bindings and a hand-written implementation using the standard MPI interface. Our evaluation shows that the proposed abstraction achieves performance comparable to the state-of-the-art MPI C++ bindings while allowing for a more flexible design of distributed applications. Furthermore, it enables programmers to design distributed applications without expending effort on data layout transformations or complicated MPI datatype definitions. The abstraction is also compatible with **Noarr** traversers, allowing for seamless cooperation with **CUDA** and **OpenMP** traversers.

Acknowledgments. This paper was supported by the Johannes Amos Comenius Programme (P JAC, Natural and anthropogenic georisks) project CZ.02.01.01/00/22_008/0004605, by Charles University institutional funding SVV (grant 260821), and Charles University Grant Agency (grant 269723).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Avans, C.N., Ciesko, J., Pearson, C., Suggs, E.D., Olivier, S.L., Skjellum, A.: Performance insights into supporting kokkos views in the kokkoscomm mpi library. In: 2024 IEEE International Conference on Cluster Computing Workshops (CLUSTER Workshops), pp. 186–187. IEEE (2024)
2. Bauke, H.: MPL - A message passing library (2014-2015). <https://github.com/rabauke/mpl>
3. Demiralp, A.C., Martin, P., Sakic, N., Krüger, M., Gerrits, T.: A C++ 20 interface for MPI 4.0. arXiv preprint [arXiv:2306.11840](https://arxiv.org/abs/2306.11840) (2023)
4. Gregor, D., Troyer, M.: Boost. mpi. MPI, November (2006)
5. Grosser, T., Groesslinger, A., Lengauer, C.: Polly: performing polyhedral optimizations on a low-level intermediate representation. *Parallel Processing Letters*, p. 1250010 (2012)
6. ISO/IEC JTC1/SC22/WG21: working draft, standard for programming language c++. Tech. Rep. N4950 (2023). <https://open-std.org/jtc1/sc22/wg21/docs/papers/2023/n4950.pdf>, revises: N4944
7. Klepl, J., Šmelko, A., Rozsypal, L., Kruliš, M.: Abstractions for C++ code optimizations in parallel high-performance applications. *Parallel Comput.* p. 103096 (2024)
8. Message passing interface forum: MPI: a message-passing interface standard version 4.1 (2023). <https://www.mpi-forum.org/docs/mpi-4.1/mpi41-report.pdf>
9. MPICH Team: MPICH v4.3.0 Manpages (2025). <https://www.mpich.org/static/docs/v4.3.0/>
10. Namjoshi, K.S., Singhanian, N.: Loopy: programmable and formally verified loop transformations. In: *International Static Analysis Symposium*, pp. 383–402. Springer (2016)
11. NVIDIA: CuTe layout documentation (2025). https://github.com/NVIDIA/cutlass/blob/main/media/docs/cpp/cute/01_layout.md, documentation on CuTe layout from the NVIDIA CUTLASS repository
12. Pellegrini, S., Prodan, R., Fahringer, T.: A lightweight C++ interface to MPI. In: 2012 20th Euromicro International Conference on Parallel, Distributed and Network-based Processing, pp. 3–10. IEEE (2012)
13. Pop, S., Cohen, A., Bastoul, C., Girbal, S., Silber, G.A., Vasilache, N.: GRAPHITE: Polyhedral analyses and optimizations for GCC. In: *proceedings of the 2006 GCC developers summit*, pp. 90–91. Citeseer (2006)
14. Pouchet, L.N., Yuki, T.: PolyBench/C 4.2.1 (2016). <https://sourceforge.net/projects/polybench>
15. Ragan-Kelley, J., Adams, A., Paris, S., Levoy, M., Amarasinghe, S., Durand, F.: Decoupling algorithms from schedules for easy optimization of image processing pipelines. *ACM Trans. Graph. (TOG)*, pp. 1–12 (2012)

16. Šmelko, A., Kruliš, M., Kratochvíl, M., Klepl, J., Mayer, J., Šimůnek, P.: Astute approach to handling memory layouts of regular data structures. In: Algorithms and Architectures for Parallel Processing: 22nd International Conference, ICA3PP 2022, Copenhagen, Denmark, October 10–12, 2022, Proceedings, pp. 507–528. Springer (2023)
17. The open MPI community: open MPI v5.0.x — Open MPI 5.0.x documentation (2025). <https://docs.open-mpi.org/en/v5.0.x/>
18. Trott, C., et al.: MDSPAN. Tech. Rep. P0009r18, WG21 (2022). <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2022/p0009r18.html>
19. Trott, C.R., et al.: Kokkos 3: programming model extensions for the exascale era. IEEE Trans. Parallel Distrib. Syst. pp. 805–817 (2021)
20. Uhl, T.N., et al.: KaMPIng: flexible and (Near) Zero-Overhead C++ Bindings for MPI. In: SC24: International Conference for High Performance Computing, Networking, Storage and Analysis, pp. 1–21. IEEE (2024)